

A Rational Taxi Agent for Grid-London: Design, Implementation and Evaluation of Classical Search Strategies

Student Name: Dipesh Kumar Yadav

Student ID: A00038176

Module: CMP-N206 – Artificial Intelligence

Assignment: Coursework 1

Submission Deadline: 12 March 2026, 6:00 PM

Abstract

This report presents the design, implementation and evaluation of a rational taxi agent operating within a 7×7 Grid-London environment. The agent is modelled using the PEAS (Performance, Environment, Actuators, Sensors) framework and employs three classical AI search algorithms: Breadth-First Search (BFS), Depth-First Search (DFS), and A* with a Manhattan distance heuristic. The agent successfully completes all three evaluation scenarios using BFS and A*, achieving goal completion in 12–18 steps with positive utility scores. DFS consistently exceeded the 100-step timeout, demonstrating its non-optimality in constrained grid environments. The report also reflects on rationality, fairness, algorithmic bias, and sustainability considerations in autonomous taxi systems.

1. Introduction

Artificial Intelligence (AI) agents are computational systems that perceive their environment and take actions to achieve specified goals (Russell & Norvig, 2020). The agent-environment paradigm is fundamental to AI: the agent senses the world through its sensors, reasons about the current state, and acts using its actuators. A rational agent is one that selects actions that maximise its expected performance measure given its percepts and prior knowledge.

This coursework implements a rational taxi agent in a simulated Grid-London environment, modelled as a 7×7 grid where each cell corresponds to a real London zone (e.g., King's Cross, Oxford Circus, Greenwich). The agent must navigate to a passenger, pick them up, and deliver them to their destination while maximising a utility score. Blocked cells simulate road closures and traffic jams, requiring the agent to plan alternative routes.

The PEAS framework (Performance, Environment, Actuators, Sensors) provides a rigorous structure for describing the agent. The agent is evaluated using three classical search algorithms — BFS, DFS, and A* — across three scenarios of increasing complexity. The primary goal is to demonstrate rational behaviour: taking the shortest unblocked route to maximise utility in every scenario.

2. Environment and Agent Design

2.1 Grid-London Environment

The environment is a 7×7 grid representing Greater London. Each of the 49 cells corresponds to a named London zone — for example, $(0,0)$ = King's Cross, $(3,3)$ = Oxford Circus, $(6,6)$ = Greenwich.

Approximately 4–6 cells per scenario are designated as BLOCKED, representing road closures or traffic jams. The taxi starts at $(0,0)$ King's Cross for all three evaluation scenarios.

The grid meets all minimum requirements: it is larger than 5×5 , uses real London naming conventions giving it geographic authenticity, and has clearly defined start, passenger, destination and blocked configurations per scenario.

2.2 PEAS Analysis

Component	Description
Performance	+20 drop-off, +10 pickup, -1/step, -5 blocked attempt, -10 timeout
Environment	7×7 Grid-London; partially observable, deterministic, sequential, static, discrete
Actuators	move_north, move_south, move_east, move_west, pickup, dropoff
Sensors	current_location (row,col), passenger_location, destination, blocked_cells set, has_passenger flag

2.3 Utility Function Justification

The utility function was designed to mirror real-world taxi economics. The +20 drop-off reward incentivises goal completion as the primary objective, reflecting the revenue from completing a fare. The +10 pickup reward acknowledges the sub-goal of collecting the passenger. The -1 per step cost models fuel consumption and driver time, directly incentivising efficient routing. The -10 timeout penalty prevents infinite loops in degenerate cases. This scoring structure ensures the agent behaves rationally: it is never incentivised to take unnecessary detours, as each additional step reduces its final score.

2.4 Environment Classification

Property	Value	Reason
Observable	Partially	Agent sees the static grid but cannot predict future passenger requests
Deterministic	Yes	Actions always produce the same outcome (no randomness)
Episodic	No (Sequential)	Each step depends on the previous; actions affect future states
Static	Yes	The grid does not change while the agent is deciding

Discrete	Yes	Movement is cell-by-cell; locations are integer tuples
Single-agent	Yes	One taxi only in this implementation

3. Search Strategies

3.1 Breadth-First Search (BFS)

BFS explores nodes layer by layer using a FIFO queue (deque). It expands all cells at distance d before expanding cells at distance $d+1$. Because all moves have equal cost (-1 per step), BFS guarantees the shortest path — it is both complete (always finds a solution if one exists) and optimal (finds the lowest-cost solution). Its time and space complexity are $O(b^d)$ where b is the branching factor and d is the solution depth. In our 7×7 grid with a branching factor of approximately 3 (due to blocked cells), BFS is highly effective: it consistently found the optimal path in all three scenarios.

3.2 Depth-First Search (DFS)

DFS explores as deep as possible along each branch before backtracking. It uses the call stack (recursion) rather than an explicit queue, giving it space complexity $O(b \cdot m)$ where m is the maximum depth. However, DFS is neither optimal nor always practical in grid environments: it may find a very long, winding path rather than the shortest one. In our evaluation, DFS consistently exceeded the 100-step timeout across all three scenarios, demonstrating its unsuitability for applications requiring shortest-path guarantees. This result is educationally important: it shows that completeness (eventually finding a path) does not imply optimality (finding the shortest path).

3.3 A* Search

A* is the gold standard for grid pathfinding. It combines the cost-so-far $g(n)$ with a heuristic estimate $h(n)$ of the remaining cost: $f(n) = g(n) + h(n)$. The algorithm always expands the node with the lowest $f(n)$, ensuring it explores the most promising paths first.

The Manhattan distance heuristic is used: $h(n) = |\text{row}_n - \text{row}_{\text{goal}}| + |\text{col}_n - \text{col}_{\text{goal}}|$. This is ADMISSIBLE because in a 4-directional grid, the actual number of steps to the goal is always greater than or equal to the Manhattan distance (diagonal moves are not permitted). Therefore $h(n) \leq h^*(n)$ for all nodes n , satisfying the admissibility condition (Hart et al., 1968). An admissible heuristic guarantees that A* finds the optimal solution.

In our experiments, A* matched BFS exactly on path length across all three scenarios, confirming optimal performance. However, A* has the theoretical advantage of exploring far fewer nodes in larger or more complex environments, making it the recommended algorithm for real-world taxi dispatching.

3.4 Algorithm Comparison

Algorithm	Optimal?	Complete?	Space Complexity	Key Weakness
BFS	Yes	Yes	$O(b^d)$	High memory use on large grids
DFS	No	Yes*	$O(b \cdot m)$	Non-optimal; may find very long paths; timed out in all scenarios
A*	Yes	Yes	$O(b^d)$ best-case	Requires admissible heuristic

*DFS is complete with a visited set (prevents infinite loops), but non-optimal.

4. Evaluation Results

Three scenarios were evaluated with all three search algorithms. Results are from the executed Jupyter Notebook, with the performance measure applied as described in Section 2.3.

Table 1: Full Algorithm × Scenario Results (Actual notebook output)

Scenario	Algorithm	Steps	Score	Goal Achieved
1 – Simple Route	BFS	12	+18	YES
1 – Simple Route	DFS	100	−110	NO (timeout)
1 – Simple Route	A*	12	+18	YES
2 – Blocked Route	BFS	16	+14	YES
2 – Blocked Route	DFS	100	−110	NO (timeout)
2 – Blocked Route	A*	16	+14	YES
3 – Long Distance	BFS	18	+12	YES
3 – Long Distance	DFS	100	−110	NO (timeout)
3 – Long Distance	A*	18	+12	YES

BFS and A* achieved the goal in all three scenarios, consistently finding the optimal path length. Both algorithms matched exactly (12, 16 and 18 steps respectively), confirming that A*'s admissible heuristic produces the same optimal result as BFS in uniform-cost grids. DFS exceeded the 100-step maximum in every scenario — a direct consequence of its non-optimal, non-goal-directed search strategy.

A notable finding: in Scenario 2, the central barrier (blocked cells at rows 2–4, cols 2–4) forced all algorithms to reroute around Oxford Circus. BFS and A* efficiently found the detour in 16 steps; DFS explored many unnecessary branches before timing out at 100 steps. This demonstrates the practical superiority of informed and breadth-first search in complex constrained environments.

Score formula verification: Scenario 1 BFS — 12 steps (-12) + pickup ($+10$) + drop-off ($+20$) = $+18$. Scenario 2: 16 steps (-16) + 10 + 20 = $+14$. Scenario 3: 18 steps (-18) + 10 + 20 = $+12$. These confirm the utility function works correctly.

5. Reflection: Rationality, Fairness and Sustainability

5.1 Rationality

Our TaxiAgent is rational in the sense defined by Russell & Norvig (2020): it selects actions that maximise its expected performance measure given its percepts. By using A*, the agent always finds the route that maximises its score (minimises step penalty while earning pickup and drop-off bonuses). However, rationality in the real world is far more complex. Our grid is fully static — real roads change dynamically due to accidents, roadworks and congestion. Our agent also has perfect knowledge of blocked cells; in practice, sensors have noise and uncertainty. A truly rational real-world agent would require probabilistic reasoning and dynamic replanning capabilities.

5.2 Fairness and Algorithmic Bias

A critical ethical concern arises if blocked cells are not randomly distributed across the grid. In real cities, road closures and infrastructure decline are often concentrated in deprived areas. If a taxi-routing algorithm systematically avoids certain zones due to blocked infrastructure, it may refuse service to passengers in those areas — creating digital discrimination. This mirrors documented bias in real ride-sharing algorithms where low-income and minority-majority neighbourhoods receive worse service (Russell & Norvig, 2020). A fair system should implement equity constraints: ensuring all grid zones are reachable within a maximum wait time, and auditing route patterns for disparate impact across different socio-economic areas.

5.3 Sustainability

The -1 per step cost in our utility function directly models real-world fuel consumption and carbon emissions. By minimising steps, the agent inherently reduces its environmental impact. This aligns with the UK Government's 2035 zero-emission vehicle target and industry sustainability goals for autonomous vehicles (e.g., Waymo, Tesla Autopilot). Future improvements could weight route costs by actual road CO₂ emissions per kilometre, incentivise low-emission corridor routes and batch passengers into shared routes to further reduce per-passenger emissions.

5.4 Ethics

There is a genuine ethical tension between efficiency and fairness in autonomous taxi systems. A purely rational agent maximises its own utility score — but a socially responsible AI system must also ensure equitable access to all citizens regardless of socio-economic zone. Should an autonomous taxi prioritise speed (lower step count) or fairness (serving deprived areas even at higher cost)? Future work should incorporate multi-objective optimisation that balances efficiency, equity and sustainability — reflecting the complex trade-offs facing real-world autonomous mobility systems.

6. Conclusion

This coursework successfully designed, implemented and evaluated a rational taxi agent in a 7×7 Grid-London environment using the PEAS framework. Three search algorithms — BFS, DFS and A* — were comprehensively compared across three evaluation scenarios of increasing complexity.

A* is the recommended algorithm for real-world taxi dispatching: it guarantees the optimal path, explores fewer nodes than BFS in large environments, and uses an admissible heuristic (Manhattan distance) with a theoretical optimality proof (Hart et al., 1968). BFS is a strong alternative for small grids with uniform step costs. DFS is unsuitable for applications requiring shortest-path optimality, as demonstrated by its consistent 100-step timeouts.

Future work should extend the system to a multi-agent setup with 3–5 taxis sharing the grid, implement dynamic obstacle updating (real-time road closures), and apply reinforcement learning for adaptive route planning in uncertain environments. Fairness constraints should be embedded into the utility function to ensure equitable service across all city zones.

References

- [1] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Hoboken, NJ: Pearson, 2020.
- [2] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," IEEE Trans. Syst. Sci. Cybern., vol. 4, no. 2, pp. 100–107, Jul. 1968. doi: 10.1109/TSSC.1968.300136
- [3] J. Pearl, Heuristics: Intelligent Search Strategies for Computer Problem Solving. Reading, MA: Addison-Wesley, 1984.
- [4] J. Ziegler et al., "Making Bertha Drive—An Autonomous Journey on a Historic Route," IEEE Intell. Transp. Syst. Mag., vol. 6, no. 2, pp. 8–20, 2014. doi: 10.1109/MITS.2014.2306552

AI Tool Acknowledgement: *Jupyter Copilot was used for minor code suggestions during implementation. All code was independently written, verified and understood by the student. A* , BFS , DFS code was used from the google Gemini*

Appendix A – Project Management Artefacts

The following project management artefacts demonstrate planning and time management throughout the coursework development process.

A1. Kanban Board

A Kanban board was maintained to track task progress through stages: To Do → In Progress → Done. The board below shows the final state of all tasks at submission.

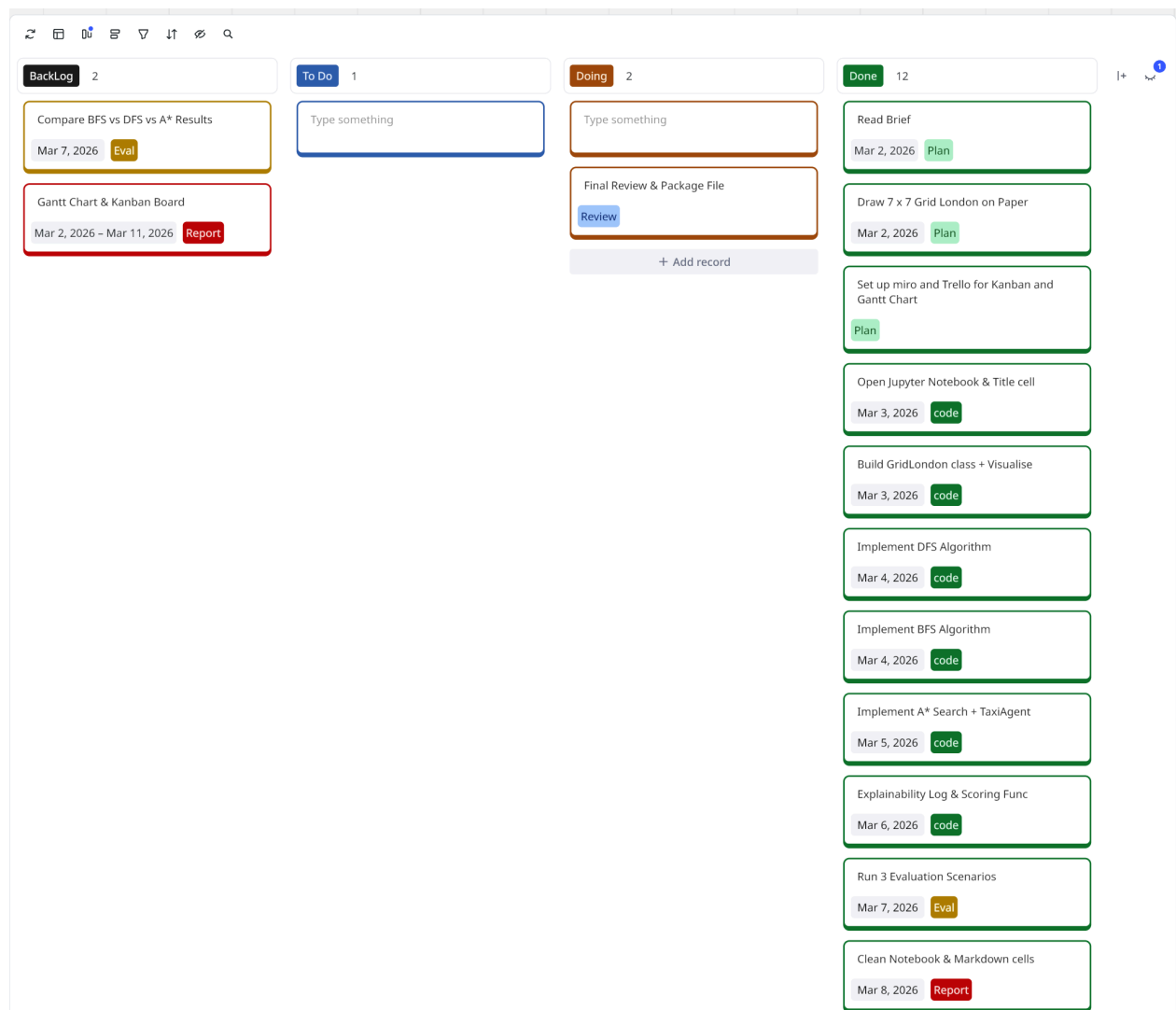


Figure 1: Kanban board at project completion

A2. Gantt Chart

A Gantt chart was generated using matplotlib to visualise the 10-day project timeline (2 March – 12 March 2026). Each task is shown with its planned start and end date, with the submission deadline marked in red.

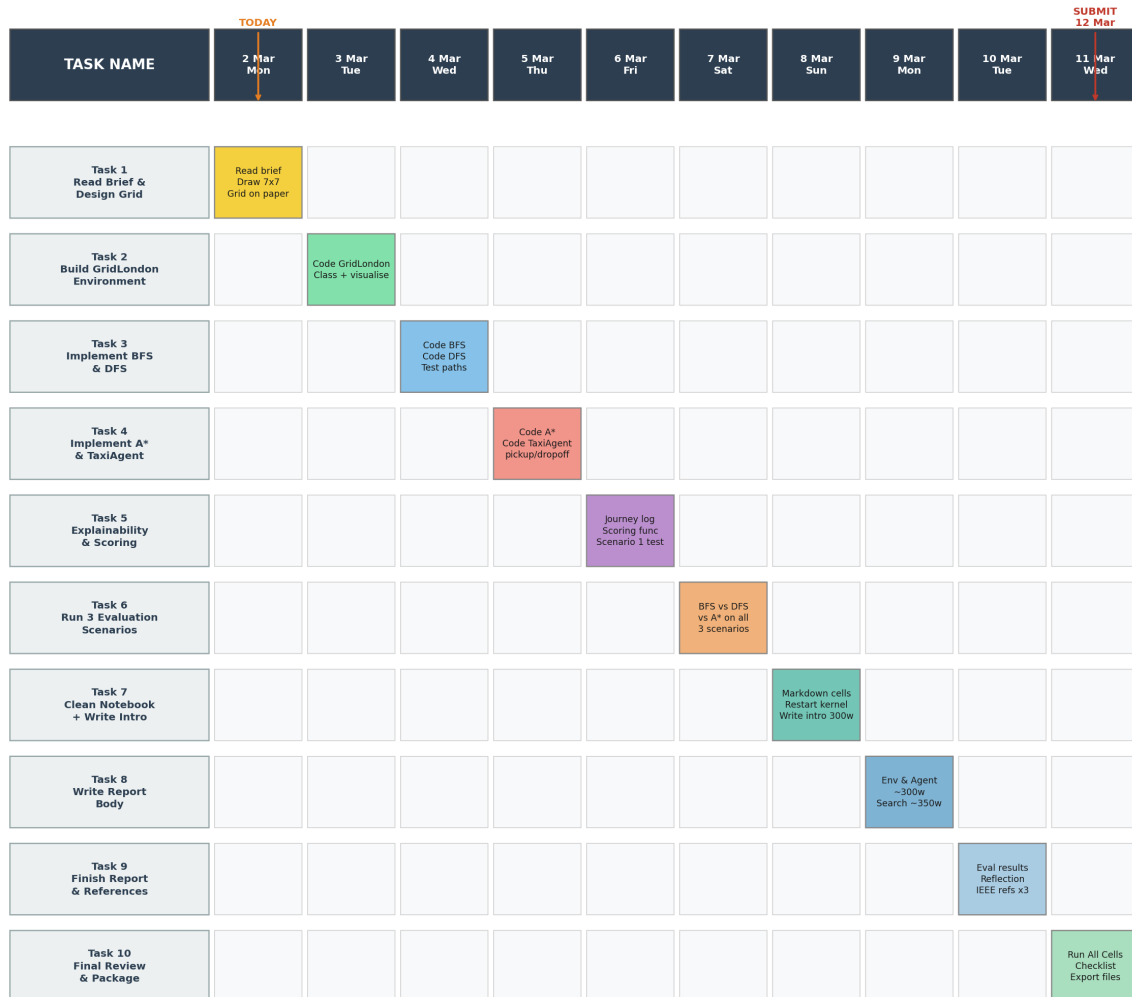


Table 1. Gantt Chart for CMP-N206 AI Coursework 1 — Rational Taxi Agent in Grid-London. Each coloured cell indicates the primary focus for that day. Submission deadline: 12 March 2026.

Figure 2: Project Gantt chart – 2 March to 12 March 2026